Node.js Backend Development

Lecture 4 — Detailed English Notes

npm • Project Initialization • package.json • Packages • Dependencies • Installing Libraries • Project Workflow

Prepared from the uploaded Hindi speech-to-text lecture transcript. Technical terms have been normalized where the intended meaning was clear. The notes explain the main concepts with examples and distinguish standard Node.js practice from transcript-specific wording.

How to study: understand the purpose of each command, run it in a test folder, and inspect the files it creates.

1. Where this lecture fits in the Node.js journey

The previous lessons introduced Node.js, running JavaScript with Node, basic JavaScript functions and callbacks, and the idea of building a backend server. This lecture moves toward managing a real Node.js project: creating a project folder, initializing it with npm, understanding package metadata, and installing reusable packages.

A backend project is more than a single JavaScript file. It usually contains source code, project metadata, installed dependencies, and scripts used to run or test the application.

2. What is npm?

npm is the standard package manager commonly used with Node.js. It helps developers discover, install, update, and manage reusable JavaScript packages. npm also provides commands for initializing projects and running scripts.

What is a package?

A **package** is a reusable collection of code and metadata. A package might provide utilities, a web framework, date handling, validation, or many other features. Instead of writing every common feature from scratch, a project can install a package and use its API.

Example

Express is a popular package used to build web servers and APIs. Utility packages can provide ready-made functions for arrays, strings, objects, and other everyday tasks.

npm is the tool used to manage packages; Node.js is the JavaScript runtime. They are related, but they are not the same thing.

3. Check that Node.js and npm are installed

Open a terminal in VS Code or PowerShell and run:

```
node --version
npm --version
```

Each command should print a version number. If either command is not recognized, the installation or PATH configuration may need attention. Node.js installers commonly include npm.

The exact version numbers may differ between computers. A different version number does not automatically mean that something is wrong.

4. Create a project folder

Create a separate folder for the code belonging to one project. Open that folder in VS Code before running project commands.

```
mkdir npm-practice
cd npm-practice
```

These commands create a folder named `npm-practice` and move the terminal into it. If you already created a folder using File Explorer, open its terminal directly instead.

Keeping each project in its own folder helps separate its code, dependencies, configuration, and Git history from other projects.

5. Initialize a Node.js project with npm init

A project is commonly initialized with `npm init`. npm asks for project details and writes them into a file named `package.json`.

```
npm init
```

The interactive command may ask for fields such as package name, version, description, entry point, test command, repository, keywords, author, and license. Press Enter to accept defaults when appropriate; you can edit the file later.

Quick initialization with npm init -y

```
npm init -y
```

The `-y` option accepts the default answers automatically. This is convenient for a practice project when you do not need to answer each prompt.

The name and other values generated by npm are project metadata. They do not create a server by themselves; you still need to write the application code.

6. Understanding package.json

`package.json` is the main metadata and configuration file for an npm project. It describes the project and records information such as dependencies and scripts.

```
{
  "name": "npm-practice",
  "version": "1.0.0",
  "description": "A Node.js practice project",
  "main": "index.js",
  "scripts": {
    "start": "node index.js"
  },
  "author": "Aryan Yadav",
  "license": "ISC"
}
```

Common fields

• **name:** the package/project name; npm package names have naming rules.

• **version:** the current project version, often written using semantic-versioning conventions such as 1.0.0.

• **description:** a short explanation of the project.

• **main:** the traditional entry-point field for a package; its role depends on how the project is used.

• **scripts:** named commands that can be run using npm, for example `npm start`.

• **dependencies:** packages needed by the application at runtime.

• **devDependencies:** packages used mainly during development, such as testing tools or file watchers.

• **author/license:** optional project metadata.

Do not assume every field must be filled in immediately. A small practice project can begin with only the fields npm creates and add others as needed.

7. Installing a package

Use `npm install` followed by the package name. For example, to install Express:

```
npm install express
```

npm downloads the package and its required dependencies, stores installed package files in `node_modules`, and normally records Express under `dependencies` in `package.json`.

A package is installed into the project when you run the command from that project's folder. Run the command in the wrong directory and you may change a different project's dependencies.

8. What is node_modules?

`node_modules` is the directory where npm places installed package code and the packages those packages depend on.

• It can contain many files and nested package folders.

• It is usually large and can generally be recreated by installing the dependencies again.

• Do not manually edit package files inside it unless you specifically understand the consequences.

For most GitHub projects, `node_modules` should not be committed. Instead, commit the dependency information and lockfile so another developer can reinstall packages.

9. package.json vs package-lock.json vs node_modules

package.json

Describes the project and the dependency ranges it declares. It is an important file to commit to Git.

package-lock.json

Records the resolved dependency tree and exact package versions chosen during installation, along with integrity and related metadata. It helps different installs reproduce a consistent dependency tree. Commit it for an application.

node_modules

Contains the installed package files themselves. It is normally regenerated with npm rather than stored in Git.

Easy memory trick: package.json = what the project declares; package-lock.json = the resolved installation record; node_modules = the downloaded files currently installed.

10. Installing development-only tools

Some packages are needed while developing but are not needed as runtime application dependencies. Install those with the development flag:

```
npm install --save-dev nodemon
```

This adds Nodemon to `devDependencies`. Nodemon watches files and restarts a Node process when relevant changes are detected. It is useful for local development.

```
npx nodemon index.js
```

If Nodemon is installed in the project, `npx` can run the local executable. You can also define a script in package.json:

```
"scripts": {
  "start": "node index.js",
  "dev": "nodemon index.js"
}
```

Then run `npm run dev`. If a scripts section already exists, add or edit the relevant entries rather than deleting unrelated scripts.

11. Using an installed package in JavaScript

After installing a package, import or require it according to the package's supported module system. For example, in a CommonJS project:

```
const express = require("express");
const app = express();
```

The package must be installed in the project or otherwise available to Node's module resolution. If Node reports that it cannot find a module, check the package name, the terminal's current folder, and whether installation succeeded.

Modern JavaScript projects may use ES module syntax such as `import`; the correct syntax depends on project configuration and the package.

12. Utility libraries and why packages are useful

The lecture discusses the idea of using existing packages instead of writing every utility yourself. A utility library may provide tested functions for common operations on arrays, strings, objects, and collections.

For example, Lodash is a JavaScript utility library. If a project needs a function provided by a package, install that package and use the documented API. Do not install a library just because it exists; first check whether native JavaScript already solves the problem clearly.

```
npm install lodash
```

```
const _ = require("lodash");

console.log(_.uniq([1, 2, 2, 3]));
// [1, 2, 3]
```

```
console.log(_.isString("hello"));
// true
```

In this example, `uniq` returns an array with duplicate values removed, and `isString` checks whether a value is a string. These examples illustrate the package workflow; use the library's documentation for the complete API.

13. Updating and removing packages

Install or update within the project's declared range

```
npm install
```

Running `npm install` in an existing project installs dependencies described by the project files. It may update the lockfile when required by the manifest or environment.

Install a specific version

```
npm install express@4
```

The version specifier controls which version range npm installs. Check the package documentation and compatibility before selecting versions.

Uninstall a package

```
npm uninstall lodash
```

This removes the package from the project and updates the dependency metadata. Use the exact package name.

14. A beginner-friendly end-to-end workflow

```
mkdir my-node-project
cd my-node-project
npm init -y
npm install express
npm install --save-dev nodemon
```

Create an `index.js` file:

```
const express = require("express");
const app = express();

app.get("/", (req, res) => {
  res.send("My first Node.js server");
});

app.listen(3000, () => {
  console.log("Server running at http://localhost:3000");
});
```

Add a development script to package.json:

```
"scripts": {
  "start": "node index.js",
  "dev": "nodemon index.js"
}
```

Run the server with `npm run dev` and visit `http://localhost:3000` in your browser. The script assumes Express and Nodemon were installed successfully.

15. Common errors and fixes

npm is not recognized

Check Node.js installation and PATH. Close and reopen the terminal after installing Node.js.

Cannot find module

Confirm the package was installed in the current project and that the package name in require/import is correct.

npm init creates a different project

Check the current directory using `pwd` in Git Bash or `Get-Location` in PowerShell. Run npm commands from the intended project folder.

package.json has invalid JSON

JSON requires double-quoted keys and string values, and it does not allow trailing commas or comments. Use valid JSON syntax.

Too much content in node_modules

This is expected for many dependencies. Keep node_modules out of Git using a .gitignore entry and reinstall dependencies when needed.

16. GitHub-ready project hygiene

For a Node.js project, a basic .gitignore can include:

```
node_modules/
.env
.DS_Store
```

Do not commit secrets such as API keys or passwords. For lecture notes alone, PDFs can be committed directly. For a code project, commit source files, package.json, package-lock.json, and the .gitignore, but usually not node_modules.

When another developer clones your repository, they can run `npm install` to recreate dependencies based on the package files.

17. Practice tasks

• Create a folder and initialize it using npm init; inspect the resulting package.json.

• Repeat using npm init -y and explain how the experience differs.

• Install Express and identify where it appears in package.json.

• Install Nodemon as a development dependency and run a server with npx nodemon.

• Compare package.json, package-lock.json, and node_modules.

• Install Lodash, use uniq on an array with duplicate values, and test isString.

• Uninstall a package and inspect the changes to the project files.

• Create a .gitignore file that excludes node_modules and explain why.

18. Quick revision sheet

• **Node.js:** runtime for executing JavaScript outside the browser.

• **npm:** package manager and project-script runner commonly used with Node.js.

• **npm init:** interactively creates package.json; **npm init -y** accepts defaults.

• **package.json:** project metadata, scripts, and declared dependencies.

• **dependencies:** packages needed at runtime.

• **devDependencies:** packages mainly used during development.

• **package-lock.json:** records resolved dependency versions/tree.

• **node_modules:** installed package files; usually excluded from Git.

• **npm install package:** installs a package in the current project.

• **npm uninstall package:** removes a package from the project.

• **Nodemon:** restarts a Node app when watched files change.

• **Lodash:** example of a utility library that provides reusable functions.

19. Transcript and scope note

The uploaded transcript is Hindi speech-to-text and contains recognition errors in English technical terms. These notes normalize likely terms such as Node.js, npm, package.json, Express, and package installation, and explain them in clear English. The examples are educational and may be cleaned up for readability. Where a transcript detail was ambiguous, the notes emphasize standard Node.js practice instead of inventing an exact quote.